Residual Blocks in Deep Neural Networks

(Keras and PyTorch Implementation)

Residual blocks are the building units of **ResNet (Residual Network)**. They introduce a shortcut (skip connection) that allows the network to learn residual mappings instead of direct mappings.

$$\text{Output} = F(x) + x$$

where $F(x)$ is the learned transformation (two Conv-BN-ReLU layers).

Two variants exist:

1. **Identity Block (left diagram):** Input and output dimensions match.
2. **Projection Block (right diagram):** Dimensions differ, requiring a 1×1 Conv on the skip path.

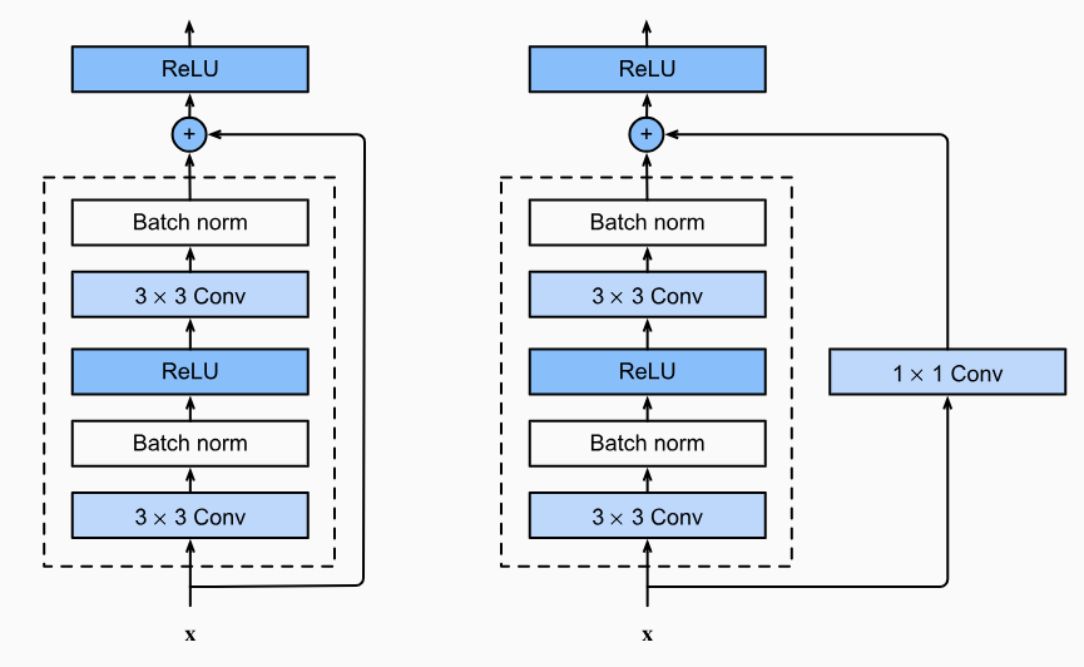

Figure 1: Residual block types: identity (left) and projection (right).

2. Keras Implementation

Listing 1: Keras Residual Block

```
import tensorflow as tf
from tensorflow.keras import layers

def residual_block(x, filters, stride=1, use_projection=False,
    name=None):
    in_channels = x.shape[-1]
    shortcut = x
```

```
    # First convolution
    y = layers.Conv2D(filters, 3, strides=stride, padding="same",
                      use_bias=False)(x)
    y = layers.BatchNormalization()(y)
    y = layers.ReLU()(y)

    # Second convolution
    y = layers.Conv2D(filters, 3, strides=1, padding="same",
                      use_bias=False)(y)
    y = layers.BatchNormalization()(y)

    # Projection if needed
    proj_needed = use_projection or (stride != 1) or (in_channels
        != filters)
    if proj_needed:
        shortcut = layers.Conv2D(filters, 1, strides=stride,
                                 padding="same", use_bias=False)(
                                    shortcut)
        shortcut = layers.BatchNormalization()(shortcut)

    out = layers.Add()([y, shortcut])
    out = layers.ReLU()(out)
    return out

# Example
inputs = tf.keras.Input(shape=(224, 224, 64))
x = residual_block(inputs, 64)
x = residual_block(x, 128, stride=2, use_projection=True)
model = tf.keras.Model(inputs, x)
model.summary()
```

3. PyTorch Implementation

Listing 2: PyTorch Residual Block

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1,
       use_projection=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3,
                               stride=stride, padding=1, bias=
                                  False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3,
                               stride=1, padding=1, bias=False)
```

```
        self.bn2 = nn.BatchNorm2d(out_channels)

        proj_needed = use_projection or (stride != 1) or (
            in_channels != out_channels)
        if proj_needed:
            self.proj = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride=
                    stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )
        else:
            self.proj = nn.Identity()

    def forward(self, x):
        identity = x
        out = self.conv1(x)
        out = self.bn1(out)
        out = F.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        identity = self.proj(identity)
        out += identity
        return F.relu(out)

# Example usage
x = torch.randn(1, 64, 224, 224)
block1 = BasicBlock(64, 64)
block2 = BasicBlock(64, 128, stride=2, use_projection=True)
y1 = block1(x)
y2 = block2(y1)
print(y1.shape, y2.shape)
```

4. Key Notes

- The skip connection prevents vanishing gradients by enabling identity mapping.
- The projection block (1×1 Conv) adjusts spatial or channel dimensions.
- Batch Normalization stabilizes learning and improves convergence.
- Both implementations use post-activation design (ReLU after addition).